

Algo de circuitos integrados de normal & union teoria
calculando el fan-out etc...

FPGA si son reconfigurables

Signal's VHDL → Tipo text

¿Dispositivos que tiene una hw configurable?

¿Codigo con errores → para corregir?

Tema 3

1. Encontrar los errores de sintaxis en el siguiente fichero fuente

```
entity muchosErrores is port
  a: bit_vector(3 to 0);
  b: out std_logic_vector(0 to 3);
  c: in bit_vector(5 downto 0);
end muchosErrores;
architecture no_correcta of muchosErrores
  signal reg: std_logic_vector(7 downto 0);
begin
  etiqueta: process
  begin
    if c = x"F" then
      b <= a;
    else
      b <= '0101';
    end if;
  end process;
  reg := reg sll 2;
end no_correcta;
```

Falta un paréntesis

(0 to 3) o (3 downto 0)

?

if c = "00" & x"F" then

línea 1
port (

línea 2
(0 to 3)
or
(0 downto 3);

línea 4
(5 downto 0);

línea 5
end muchosErrores;

línea 6
architecture no_correcta of muchosErrores is

línea 7
signal reg;

línea 9
process(c)

línea 11
if c = "00" & x"F" then

línea 12
b <= a;

línea 14
b <= "0101";

$\left. \begin{array}{l} \text{std} \\ \text{std} \\ \text{ror} \\ \text{rol} \end{array} \right\} \text{ hit_vector} \rightarrow \text{solo se puede hacer con variable de dicho tipo}$

añadir al principio use IEEE.numeric_std.all

línea 7

signal reg: std_logic_vector ...
 reg <= std_logic_vector(shift_left(unsigned(reg), 2));
 signal reg: hit_vector (7 downto 0);

línea 18

end no_correcto

2. Escribir la declaración de la entity para la siguiente architecture, considerando que todas las signals de la architecture son ports.

```

architecture escribe_entity of problema is
begin
  decod_memo: process(addr)
  begin
    shadow_ram_sel <= '0';
    sram_sel <= '0';
    if addr >= x"0100" and addr < x"4000" then
      shadow_ram_sel <= '1';
    elsif addr >= x"8000" and addr < x"C000" then
      sram_sel <= '1';
    end if;
    promsel <= '0';
    if mem_mapped = '0' and bootup = '1' then
      prom_sel <= '1';
    end if;
  end process decod_memo;
  mem_mapped <= shadow_ram_sel or sram_sel;
end escribe_entity;
  
```

entity problema is port (

addr: in std_logic_vector(15 downto 0);

mem_mapped: in std_logic;

bootup: in std_logic;

```
Shadow_ral_sel : inout std_logic;
sram_sel : inout std_logic;
pram_sel : out std_logic;
end problema
```

3. ¿Cuál es la ventaja de usar la asignación siguiente para el array a?

```
architecture problema of asignacion
is
  signal a : std_logic_vector(31 downto 0);
begin
  a <= (others => '1');
  ...
end problema;
```

Que lo pone todo a 1 (te evitas de poner los 32 bits a 1)

```
entity asignacion is port(
  generic (n : integer := 32);
```

```
architecture .....
```

```
  signal a : std_logic_vector(n-1 downto 0);
  a <= (others => '1');
```

Problema -> máquina estado / Puede venir con hacer jerarquía
Unidad lógica y ra

4. Determinar el valor de cada bit de los arrays del siguiente segmento de código VHDL.

```
Variable A: std_logic_vector(0 to 3) := (3 => '1', Others => '0');  
Variable B: std_logic_vector(6 downto 3);  
Variable C: std_logic_vector(7 downto 0) := X"5A";  
Variable D: std_logic_vector(0 to 7);
```

```
-----  
B := "0110";  
A := B;  
-- se usa el valor actualizado de A  
C(5 downto 0) := A(0 to 1) & B;  
D := B & C(7 downto 4);  
A := '1' & B(6) & "01";  
-----
```

variable A: std_logic_vector(0 to 3) := (3 => '1', others => '0');

:= "0001";

0 1 2 3 → sentido ascendente

-- A := "0001"

variable B: std_logic_vector(6 downto 3);

variable C: std_logic_vector(7 downto 0) := X"5A"

-- C := "0101 1010"

7 6 5 4 3 2 1 0
(5) (A)

Se deja igual

variable D: std_logic_vector(0 to 7);

B := "0110";

A := B -- Ya no vale 6 de ante vale, 6 de B → A := "0110"

C(5 downto 0) := A(0 to 1) & B; -- C := "0101 0110"

Los 6 bits nuevos
significativos

0 1 2 3
7 6 5 4 3 2 1 0
Concatenar con B

D := B & C(7 downto 4);

-- D := "0110 0101"

15 bits concatenados de bits 7 al 4 de C

A := 'S' & B(6) & "01";

-- A := "5005";

Varios bit: " " / Único bit: ' '

5. Declarar la matriz de celdas de una ROM de organización 16 x 8 con el contenido de la siguiente tabla.

Dirección	Contenido	Dirección	Contenido
0	0	8	87 ₁₆
1	3 ₁₆	9	44 ₁₆
2	45 ₁₆	A	B9 ₁₆
3	EF ₁₆	B	AA ₁₆
4	55 ₁₆	C	FF ₁₆
5	2C ₁₆	D	58 ₁₆
6	79 ₁₆	E	9D ₁₆
7	AD ₁₆	F	66 ₁₆

architecture functional of Rom is

type matriz is array (0 to 15) of std_logic_vector (7 downto 0);

Constant memoria : matriz := (X"00", X"03", X"45",
X"EF", X"55", X"2C", X"79", X"AD", X"87",
X"44", X"B9", X"AA", X"FF", X"58", X"9D", X"66");

6. Dada la declaración de las siguientes señales, calcúlese el resultado de aplicar los operadores lógicos al contenido inicial de dichas señales

Signal oper_A : std_logic_vector(7 downto 0) := "10011010";

Signal oper_B : std_logic_vector(7 downto 0) := "11010110";

Signal Resul : std_logic_vector(7 downto 0);


```

Resul <= not oper_A;
Resul <= oper_A and oper_B;
Resul <= oper_A xor oper_B;
Resul <= oper_A nor oper_B;

```

oper_A := "00011010" not

Resul <= not oper_A; -- Resul: "11100101"

Resul <= oper_A and oper_B; Sumar

oper_A := "00011010"

oper_B := "11011010"

Resul: "00011010"

x	y	or	nor
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

Resul <= oper_A xor oper_B -- Resul: "01001100"

Resul <= oper_A nor oper_B -- Resul: "00100001"

7. Dada la declaración de la siguiente señal, calcúlese el resultado de aplicar los operadores de desplazamiento al contenido inicial de dicha señal

Signal registro : bit_vector(7 **downto** 0) := "10011010";

```

-----
registro <= registro sll 2;
registro <= registro srl 4;
registro <= registro rol 4;
registro <= registro ror 5;

```

registro := "10011010" desplaza. log. a la izq

registro <= registro sll 2; -- registro: "01101000" ↑
desplaza. log. a la dere 10011010

registro $\leftarrow$ registro srl 4; -- registro := "0000 800 8"
rotate a la izquierda

registro $\leftarrow$ registro ror 4 -- registro := "50505 005"
rotate a la derecha

registro $\leftarrow$ ror 5 -- registro := "550050 500"
rotate a la derecha